# Image to 3D Interactive Worlds

Adyasha Patra Gaurav Joshi Jay Chaudhary Keshav Gupta

## 1. Introduction

Generating simulation-ready 3D environments from sparse visual inputs, such as a single image, is a fundamental challenge at the intersection of computer vision and embodied AI. In this project, we propose an *agentic framework* to construct interactive, physically grounded 3D scenes from minimal observations, with a primary focus on image-based generation.

Building on recent advances such as [1], which reconstructs amodal, camera-centric 3D environments from a single image, our approach moves toward *agent-driven environment generation*. Specifically, we employ an agentic architecture in which modular agents are responsible for perception, physical property inference, and simulation refinement. Leveraging prior work in inverse rendering and 3D Gaussian Splatting, these agents collaboratively recover not only geometry and appearance but also physically meaningful properties such as mass, friction, and elasticity.

The generated environments are simulated using Material Point Methods (MPM), enabling physically consistent interactions. Crucially, agents are not only deployed within these environments but are also involved in constructing and validating them, iteratively refining scene representations through interaction and feedback. The resulting digital scenes serve as dynamic testbeds where autonomous agents can perceive, reason, and act within physically grounded environments.

A key limitation in current embodied AI and reinforcement learning research is the reliance on manually engineered simulation environments. While such environments are effective for narrow domains, they do not scale to the diversity and complexity of real-world scenarios.

Automating the generation of simulation-ready environments from visual data offers a path toward scalable training and evaluation of agentic systems. By transforming real-world images into interactive 3D worlds, we can expose agents to a wide distribution of physical scenarios without manual environment design.

This paradigm enables agents to learn adaptive and hierarchical behaviors in visually rich and physically realistic settings, bridging the gap between passive visual understanding and active physical reasoning.

Our main contributions are as follows:

- We propose a modular agentic image-to-simulation framework that converts a single RGB image and natural language prompt into a structured, physically grounded 3D scene.
- We design specialized agents for semantic relevance, segmentation, 3D reconstruction, material-property inference, force initialization, camera placement, background construction, and Blender-based execution.
- We produce editable Blender simulation scripts rather than only rendered videos, allowing users to modify physical parameters such as mass, friction, damping, collision shape, and initial force before re-simulation.
- We evaluate the system against strong video-generation baselines using photorealism and prompt-alignment metrics, showing that although our current renders are less photorealistic, the generated scenes achieve competitive alignment with the intended prompts.

## 2. Related Works

**Image-to-3D Physical Scene Generation.** [1] demonstrates that a single RGB image can be lifted into a physically interactive 3D environment by combining semantic scene understanding with physics-based simulation. It estimates geometry, object layouts, and material properties, enabling controllable simulations under varying conditions.

**Simulation-Ready Reconstruction from Video.** [5] formulates reconstruction as an energy-based optimization problem, producing a complete 3D scene graph with physical constraints from video input. While powerful, it relies on multi-view observations, motivating our focus on single-image generation.

**Neural Rendering and Physical Parameter Estimation.** Recent advances in inverse rendering and neural scene representations ([3], [2]) enable the recovery of 3D structure and appearance from 2D observations. Extending these approaches to infer physical parameters is critical for enabling downstream interaction and simulation. Works such as [4] extend this line of research by leveraging strong 2D foundation models to lift images into structured 3D representations, enabling more accurate object decomposition, geometry estimation, and scene understanding from sparse in-

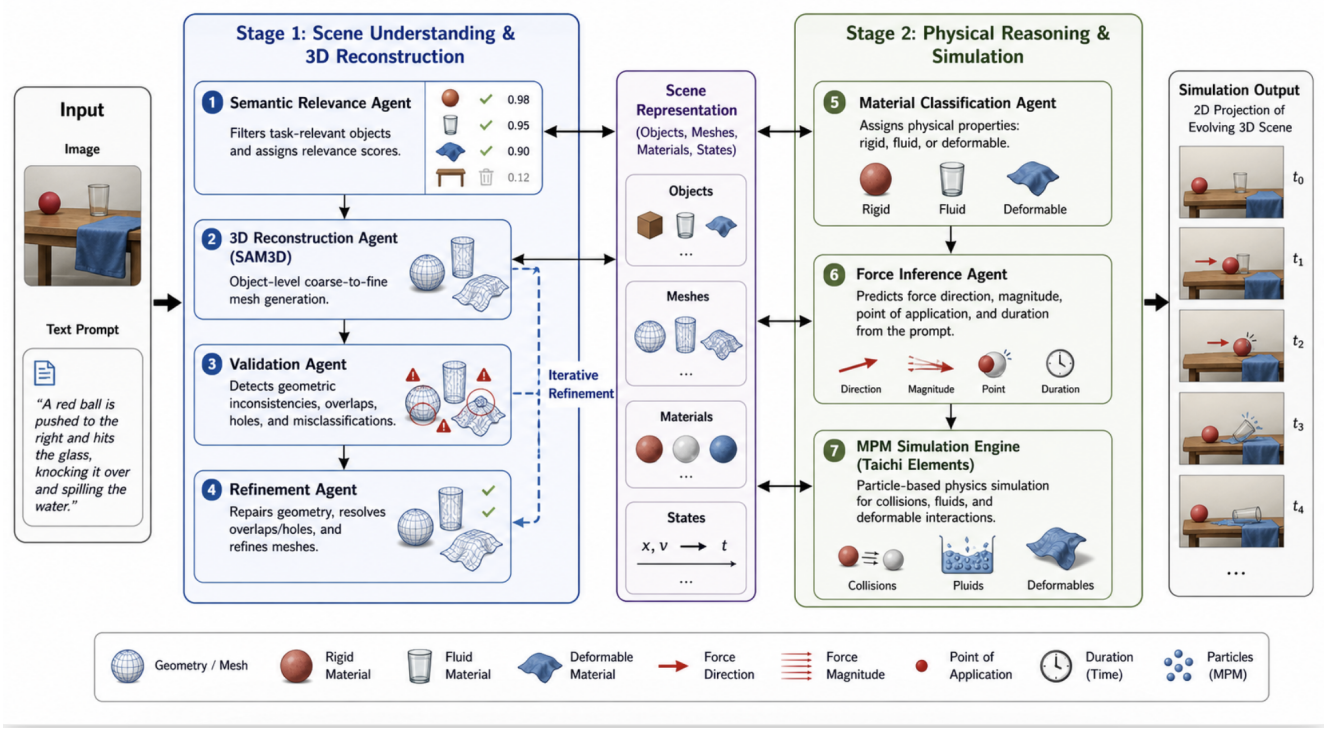


Figure 1. Overview illustration of the proposed agentic image-to-simulation pipeline.

puts.

## 3. Method Overview

We propose a multi-agent framework that transforms an input image and natural language prompt into a physically grounded Blender simulation. The system combines semantic understanding, segmentation, 3D reconstruction, physical reasoning, and scene composition to automatically generate a structured scene representation that can be executed inside Blender.

The framework is organized into three primary stages: (1) perception and scene understanding, (2) physical reasoning, and (3) scene composition and simulation execution. A key design principle of the system is modularity, where specialized agents communicate through a centralized structured scene representation.

In the first stage, the system performs semantic grounding and object-level scene understanding. Given an input image and prompt, the Semantic Relevance Agent identifies objects relevant to the described interaction while filtering out irrelevant scene elements. Rather than segmenting every object in the image, the system selectively focuses only on entities referenced or implied by the prompt, reducing unnecessary computation and improving downstream reasoning. For each selected object, the agent invokes SAM2 to generate high-quality segmentation masks.

The segmented objects are then passed to the 3D Reconstruction Agent, which uses a SAM3D-based reconstruction pipeline to generate object-level meshes together with their spatial properties, including object positions, orientations, and approximate geometric structure. These reconstructed meshes form the geometric foundation of the simulation scene.

In the second stage, the framework augments the reconstructed scene with physical properties and dynamic behavior. The Material Classification Agent predicts physically meaningful simulation parameters for each object, including mass, friction, restitution, damping coefficients, collision shapes, and rigid-body configuration. Instead of assigning only semantic material labels, the agent produces Blender-compatible physical attributes that can be directly integrated into the simulation pipeline.

Next, the Force Inference Agent interprets the prompt in the context of the reconstructed scene and estimates the initial dynamic state of the objects. This includes parameters such as initial linear velocity, angular velocity, and motion direction, enabling the system to translate high-level language instructions into physically executable motion initialization. For example, in the prompt "Drop the teddy bear and throw plushy towards the teddy," the system must identify the teddy bear and plushy as separate interactive objects, assign the teddy a downward motion, and initialize the plushy with a directed velocity toward the teddy, as il-

lustrated in Figure 3.

Throughout the pipeline, the system maintains a centralized structured scene representation. This representation stores semantic labels, segmentation outputs, reconstructed meshes, object transformations, physical properties, and simulation parameters in a unified JSON-based format. An example object entry is shown below:

```
{
  "objects": {
    "object_0": {
      "label": "briefcase",
      "mass": 2.0,
      "friction": 0.5,
      "restitution": 0.1,
      "linear_damping": 0.04,
      "angular_damping": 0.1,
      "collision_shape": "CONVEX_HULL",
      "initial_linear_velocity": [
        0.0, 0.0, 0.0
      ],
      "initial_angular_velocity": [
        0.0, 0.0, 0.0
      ]
    }
  }
}
```

In the final stage, the system performs scene composition and simulation setup inside Blender. The Camera Agent automatically determines camera placement and viewing angles to ensure visibility of important scene interactions while minimizing occlusions. In parallel, the Background and Lighting Agent generates a lightweight environmental context, including floor geometry, sky setup, and lighting configuration, to provide visually coherent environmental grounding.

The final structured scene representation is passed to the Blender Execution Engine through a Python-based execution pipeline. The engine instantiates reconstructed meshes, assigns rigid-body properties, configures environmental settings, initializes object dynamics, and executes simulation and rendering using the Blender Python API.

The current implementation primarily focuses on rigid-body simulation using Blender's default physics system. However, the modular architecture naturally supports future extensions involving fluid, cloth, and soft-body simulation systems. Additionally, the framework is designed to incorporate a future feedback-driven refinement module that evaluates rendered frames and simulated motion for physical plausibility, material consistency, collision behavior, and interaction correctness.

### 3.1. Agent Roles

The proposed framework consists of multiple specialized agents, each responsible for a distinct aspect of perception, reasoning, scene construction, or simulation execution. This modular decomposition improves interpretability, scalability, and extensibility while enabling individual components to be independently improved or replaced.

**Semantic Relevance Agent.** Identifies objects relevant to the interaction described in the prompt and filters out unrelated scene elements. The agent jointly analyzes the image and text prompt, then invokes SAM2 to generate precise segmentation masks for selected objects. This agent is responsible for deciding which parts of the image should become active simulation entities and which should be treated as static background context. For example, in a prompt involving boxes falling near a child, the boxes are selected as dynamic objects while the child may be treated as a semantic reference or static obstacle depending on the intended interaction. This selective filtering reduces unnecessary reconstruction work and helps prevent cluttered simulations with irrelevant objects.

The output of this agent is a set of object labels, masks, and relevance scores that are stored in the centralized scene representation. These outputs provide the semantic grounding for all downstream modules. Because errors at this stage propagate through the entire pipeline, the agent also plays an important validation role by checking whether the selected objects are consistent with the prompt and whether the masks are sufficiently complete for object-level reconstruction.

**3D Reconstruction Agent.** Generates object-level meshes from segmented objects using a SAM3D-based reconstruction pipeline. In addition to geometry reconstruction, the agent estimates spatial properties such as object positions, orientations, and approximate scene layout. Its goal is not only to produce visually recognizable meshes, but also to recover geometry that can be used by the physics engine. Therefore, the agent converts segmented image regions into object-level assets with approximate scale, pose, and placement relative to other entities in the scene.

Since single-image reconstruction is inherently ambiguous, the agent relies on geometric priors and semantic cues to infer missing surfaces and approximate object volume. The resulting meshes may be simplified into collision-friendly proxies such as cuboids, convex hulls, or low-resolution meshes when exact reconstruction is unreliable. This tradeoff improves simulation stability while preserving the spatial arrangement needed for physically meaningful interactions.

**Material Classification Agent.** Predicts simulation-ready physical attributes for reconstructed objects, including mass, friction, restitution, damping coefficients, collision shapes, and rigid-body configuration. The generated

parameters are directly compatible with Blender's physics engine. Rather than only assigning a semantic material label such as "wood," "metal," or "cardboard," this agent translates visual and textual evidence into numerical values that determine how objects behave during simulation.

For each object, the agent estimates whether it should be dynamic, passive, or constrained. It also assigns approximate density and contact properties based on object category, apparent size, and expected material. These estimates are crucial for producing plausible interactions: a cardboard box should fall, slide, and collide differently from a rubber ball, a metal can, or a piece of cloth. Although the current implementation uses coarse priors, this module provides a natural interface for future learned material estimation and physics-parameter refinement.

**Force Inference Agent.** Estimates the initial dynamic state of the scene using the prompt together with reconstructed geometry and material properties. The agent predicts parameters such as initial linear velocity, angular velocity, and motion direction to initialize physically executable interactions. This component translates high-level language descriptions into concrete simulation controls. For example, a prompt describing an object being pushed, dropped, thrown, or toppled must be converted into forces or velocities that Blender can execute.

The agent reasons over object relationships and the intended event structure. It determines which objects should move first, which objects should remain stationary, and how motion should propagate through contact. In the current system, trajectories are mainly induced through initial velocities and damping parameters, but the same interface can later support continuous forces, constraints, articulated motion, and action-conditioned control policies.

**Camera Agent.** Automatically determines camera placement and scene framing to maximize visibility of important interactions while reducing occlusions. The agent can iteratively refine viewpoints based on rendered outputs. It uses the reconstructed object positions, predicted motion direction, and prompt-relevant entities to choose a camera angle that clearly shows the important physical event. This is especially useful when the generated scene contains multiple objects or when the interaction unfolds over time.

The Camera Agent also provides consistency between the input image and the rendered simulation. When appropriate, it attempts to preserve a similar viewpoint to the original observation so that the generated frames remain visually interpretable. In future versions, this module can be extended to evaluate rendered previews, detect occlusions or off-screen motion, and automatically update camera placement before final rendering.

**Blender Execution Engine.** Serves as the final orchestration component of the framework. We use Blender in place of Taichi in the current implementation because it provides an accessible interface for asset import, rigid-body simulation, material assignment, camera control, and rendering within a single environment. Using the Blender Python API, the engine imports reconstructed meshes, applies physical properties, configures the environment, initializes object dynamics, and executes simulation and rendering.

The execution engine converts the structured scene representation into an executable script. This script instantiates objects, assigns rigid-body settings, creates collision geometry, sets gravity and frame ranges, applies initial velocities, and renders the resulting motion. A key benefit of this design is editability: users can inspect the generated script, manually adjust parameters such as mass or friction, and immediately re-run the simulation. This makes the output more controllable than a generated video and better suited for iterative experimentation.

**Future Feedback and Validation Module.** A planned extension designed to evaluate generated simulations for physical plausibility, collision behavior, material consistency, and interaction correctness. This module could iteratively refine camera placement, physical parameters, and motion predictions to improve realism and simulation stability. After an initial render, the module would inspect frames or simulation logs to detect common failures such as object interpenetration, unrealistic bouncing, missing collisions, unstable stacks, or prompt-misaligned motion.

The feedback module would close the loop between generation and validation. Instead of producing a single simulation in one pass, the system could repeatedly adjust object placement, collision shapes, material parameters, and force initialization until the rendered behavior satisfies both physical and semantic constraints. This extension is central to making the framework more robust for complex scenes where single-pass inference is insufficient.

## 3.2. Implementation Details

The current system is implemented as a staged Python pipeline that passes a shared JSON scene representation between agents. Each stage reads the current scene state, appends its own predictions, and writes the updated representation for downstream modules. This design keeps the system transparent and debuggable: object masks, mesh paths, physical parameters, camera settings, and force initialization values can be inspected independently before the final Blender script is generated.

Given an input RGB image and natural language prompt, the pipeline first performs prompt-conditioned object selection. The Semantic Relevance Agent identifies the objects that participate in the requested physical event and calls the segmentation module only for those objects. The resulting masks are saved as object-level assets and indexed in the scene JSON with labels, relevance scores, and image-space

bounding boxes. These masks are then used by the reconstruction stage to produce individual 3D assets rather than a single monolithic scene mesh.

For 3D reconstruction, each segmented object is processed independently and exported as a mesh file with an associated canonical transform. Physical parameters are assigned using category-level priors combined with prompt context. The Material Classification Agent maps each object to Blender-compatible rigid-body settings, including mass, friction, restitution, linear damping, angular damping, collision shape, and active/passive state. The Force Inference Agent then converts action verbs and object relations in the prompt into initial velocities or angular velocities. For example, verbs such as "drop," "push," and "throw" are mapped to different velocity directions and magnitudes conditioned on the relative positions of the target objects.

The Blender Execution Engine converts the final scene JSON into an executable Python script that imports meshes, applies transformations, assigns physical and collision properties, constructs static supports, configures lighting and cameras, and runs the simulation. The renderer then produces key frames or video outputs. Because the script remains editable, users can modify parameters and rerun simulations without repeating the entire reconstruction pipeline. In practice, the system serves as both a generation and debugging tool. Intermediate JSON files provide stage-by-stage visibility, helping identify failures in object selection, segmentation, mesh reconstruction, physical parameter assignment, force inference, or Blender execution. This modular logging is particularly important for single-image simulation, where under-constrained scene properties can cause errors to accumulate across stages.

# 4. Results

## 4.1. Experiments

We evaluate the proposed pipeline against a set of strong video-generation and image-to-video baselines, including Runway Gen-3, MOFA-Video, Pika 1.5, Kling 1.0, DragAnything, and PhysGen3D. These methods represent widely used approaches for generating dynamic visual content from image or text conditions. In contrast to these baselines, our method produces an editable Blender scene and simulation script rather than only a rendered video. This distinction is important because the generated output can be inspected, modified, and re-simulated by changing physical parameters such as mass, friction, restitution, initial force, or object constraints.

## 4.2. Dataset

For our preliminary evaluation, we manually assess key frames across three representative scenes: a food scene, a kids-room scene, and a traffic scene. These examples are designed to test whether the pipeline can identify semantically relevant objects, reconstruct a plausible 3D arrangement, and initialize a physically meaningful simulation from sparse visual input. The comparison values for the baseline methods follow the reported evaluation protocol that uses GPT-4o as an automated judge. For our method, we manually evaluate the generated key frames because the current system outputs structured Blender simulations whose intermediate scene representations and rendered frames require direct inspection.

Table 1. Quantitative comparison across photorealism and alignment metrics.

| Method | PhotoReal | Align |
|---|---|---|
| Runway Gen-3 | 0.896 | 0.144 |
| MOFA-Video | 0.764 | 0.304 |
| Pika 1.5 | 0.863 | 0.563 |
| Kling 1.0 | 0.874 | 0.596 |
| DragAnything | 0.756 | 0.380 |
| PhysGen3D | 0.867 | 0.796 |
| **Ours** | **0.410** | **0.600** |

## 4.3. Metrics

We report two metrics: **PhotoReal** and **Align**. PhotoReal measures the visual realism of the generated output, including appearance quality, material realism, lighting, and overall resemblance to real video frames. Align measures how well the generated scene matches the input prompt and intended scenario, including whether the correct objects are present, whether the interaction is semantically appropriate, and whether the generated motion reflects the described event. Together, these metrics capture the main tradeoff in our current system: the output is less photorealistic than neural video generators, but it remains structured, editable, and physically interpretable.

## 4.4. Quantitative Analysis

Table 1 shows that our method achieves a PhotoReal score of 0.410, which is lower than the neural video-generation baselines. This gap is expected because methods such as Runway Gen-3, Pika 1.5, and Kling 1.0 are trained on large-scale video datasets and directly optimize for visually realistic video synthesis. Our current implementation instead uses basic Blender materials, approximate geometry, and simple lighting, so visual realism is limited by texturing, illumination, and rendering fidelity. This limitation is addressable through improved material estimation, texture transfer, lighting reconstruction, and higher-quality rendering.

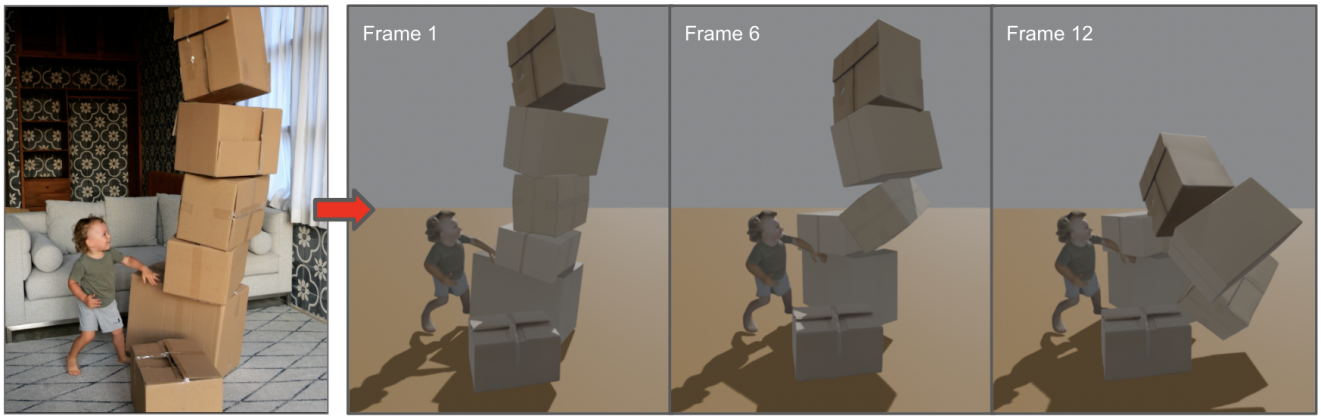


Figure 2. Qualitative result of the proposed image-to-simulation pipeline. Starting from a single image of a child standing beside an unstable stack of boxes, the system reconstructs a corresponding 3D scene and simulates the boxes as they topple over time.

Despite the lower PhotoReal score, our method obtains an Align score of 0.600, which is competitive with several baselines and close to the stronger video-generation systems. This result suggests that the agentic pipeline is able to correctly identify relevant objects, arrange them in an appropriate scene context, and initialize interactions that match the prompt. In other words, the semantic setup of the simulation is often correct even when the final rendered appearance is visually simple. The remaining errors are primarily related to physical parameter tuning, contact stability, and material realism rather than prompt understanding alone.

The key advantage of our approach is controllability. Standard video generators produce pixels, making it difficult to edit the generated physical process after synthesis. Our system instead produces a Blender-executable scene in which users can directly adjust object masses, friction coefficients, damping values, collision shapes, initial velocities, and forces before re-rendering. This makes the output more useful for simulation, debugging, and embodied-agent experimentation, even when its current photorealism is below that of large neural video models.

### 4.5. Qualitative Analysis

Figure 2 illustrates an initial qualitative result of our pipeline. Given a single RGB image containing a child and a vertical stack of cardboard boxes, the system identifies the relevant objects, reconstructs their approximate 3D geometry, and places them into a physically simulated scene. The subsequent frames show the predicted dynamics as the unstable stack begins to collapse, demonstrating that the generated environment supports both semantic reconstruction and physically meaningful temporal behavior. Although the reconstructed objects are still simplified, the example highlights the pipeline's ability to convert a static real-world observation into an interactive 3D scene with plausible motion.

Figure 3 provides an additional qualitative example for a more action-specific language prompt: "Drop the teddy bear and throw plushy towards the teddy." In this case, the pipeline successfully grounds the two target objects, assigns distinct motion roles to them, and generates a simulation in which the teddy bear drops while the plushy moves toward it. This example demonstrates that the method can go beyond passive scene reconstruction and use language to initialize directed object interactions.

More specifically, the example demonstrates how the different agents in the pipeline cooperate to produce a coherent simulation. The perception components first infer object locations, approximate depth relationships, and scene layout from the input image. These outputs are then translated into a structured scene representation where individual objects are assigned geometric proxies, physical properties, and spatial constraints. Even with coarse cuboid approximations, the system is able to recover the relative arrangement of the stacked boxes and preserve the overall physical structure of the scene.

The simulation further reveals that physically grounded interactions naturally emerge once the reconstructed objects are placed inside the Blender environment. Small perturbations and gravity cause the stack to lose stability, leading to collisions, toppling, and cascading motion across multiple frames. This behavior is significant because the motion is not explicitly scripted; instead, it arises from the interaction between the reconstructed geometry and the physics engine. As a result, the generated sequence exhibits temporal consistency and physically plausible object dynamics.

The experiment also highlights several important observations regarding the current capabilities and limitations of the framework. First, the pipeline can successfully infer a reasonable 3D arrangement from a single monocular image despite limited geometric information. Second, the use of

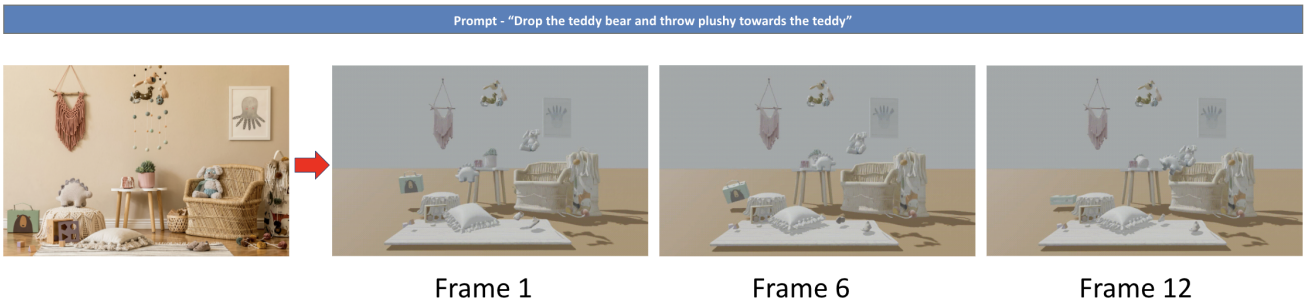


Figure 3. Qualitative result for the prompt "Drop the teddy bear and throw plushy towards the teddy." Our method identifies the relevant objects, initializes the teddy bear with a falling motion, and throws the plushy toward the teddy, producing an editable Blender simulation with prompt-aligned object dynamics.

simplified rigid-body representations allows efficient simulation while still preserving recognizable scene behavior. However, fine-grained geometric details, material properties, and accurate mass estimation remain challenging, occasionally leading to unrealistic motion or unstable interactions. In particular, highly precise contact modeling and complex object shapes are difficult to recover from a single image alone.

Overall, these preliminary results suggest that the proposed framework provides a promising foundation for automatic image-to-simulation generation. The ability to reconstruct scenes, assign physical behaviors, and generate temporally evolving interactions from minimal visual input represents an important step toward scalable and autonomous 3D world generation systems.

## 5. Challenges and Future Work

Despite these encouraging initial results, several challenges remain before the proposed framework can reliably generate complex, realistic, and simulation-ready environments from sparse visual input. A major limitation is robust object interaction, especially when multiple reconstructed objects collide with one another. Small errors in geometry, scale, object placement, or collision-shape estimation can lead to unstable contacts, interpenetration, or physically implausible motion. Future work will therefore focus on improving collision-aware scene composition and adding validation modules that can detect and correct unstable object configurations before simulation begins.

Another challenge is controlling which objects should remain fixed under gravity. In the current Blender-based setup, gravity acts globally on all rigid bodies unless objects are explicitly marked as passive or constrained. This creates difficulties for scene elements such as floors, walls, tables, or supporting structures, which should remain stationary while dynamic objects move and collide. A future extension will introduce a more explicit support-relation and constraint-inference module, allowing the system to distinguish between dynamic objects, static environmental structures, and objects that should be temporarily fixed or attached to other surfaces.

The present motion model is also limited because object trajectories are primarily determined by initial velocity, angular velocity, and damping factors. While this is sufficient for simple interactions such as falling or toppling, it restricts the range of possible behaviors and makes it difficult to represent more complex trajectories involving continuous forces, articulated motion, agent actions, or temporally varying control signals. Future versions of the system will incorporate richer force and action models, enabling language prompts to specify more expressive physical events and longer-horizon interactions.

In addition, the current implementation focuses mainly on rigid-body objects. Real-world scenes often contain diverse materials and object types, including deformable objects, cloth, granular materials, liquids, and other non-rigid structures. Extending the pipeline beyond rigid-body simulation is an important direction for future work. The modular agent architecture is well suited for this extension, since additional material-specialized agents could infer whether an object should be simulated as rigid, soft, fluid, or deformable and then select the appropriate simulation backend or physical representation.

Finally, preserving visual realism remains an open challenge. At present, the full simulation and rendering pipeline is implemented in Blender, which provides convenient tools for scene composition and rigid-body simulation but does not automatically guarantee photorealistic appearance or physically accurate material behavior. Future work will explore tighter integration between reconstructed scene appearance, lighting estimation, material modeling, and rendering refinement so that generated simulations remain visually consistent with the original input image while also supporting physically plausible interaction.

## References


[1] Boyuan Chen, Hanxiao Jiang, Shaowei Liu, Saurabh Gupta, Yunzhu Li, Hao Zhao, and Shenlong Wang. Physgen3d: Crafting a miniature interactive world from a single image. *CVPR*, 2025. 1

[2] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering, 2023. 1

[3] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis, 2020. 1

[4] SAM 3D Team, Xingyu Chen, Fu-Jen Chu, Pierre Gleize, Kevin J Liang, Alexander Sax, Hao Tang, Weiyao Wang, Michelle Guo, Thibaut Hardin, Xiang Li, Aohan Lin, Jiawei Liu, Ziqi Ma, Anushka Sagar, Bowen Song, Xiaodong Wang, Jianing Yang, Bowen Zhang, Piotr Dollár, Georgia Gkioxari, Matt Feiszli, and Jitendra Malik. Sam 3d: 3dfy anything in images, 2025. 1

[5] Hongchi Xia, Chih-Hao Lin, Hao-Yu Hsu, Quentin Leboutet, Katelyn Gao, Michael Paulitsch, Benjamin Ummenhofer, and Shenlong Wang. Holoscene: Simulation-ready interactive 3d worlds from a single video, 2025. 1